

Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação

SSC0502 - Laboratório de Introdução à Ciência de Computação I

Projeto - Uaibank

Autor:

Pedro Artur Barberá Sarni

Maio de 2026
São Carlos - SP, Brasil

Sumário

1	Introdução	2
2	Arquitetura Geral do Sistema	2
2.1	Pseudocódigo da Função Principal (Main)	2
2.2	Fluxograma da Estrutura Geral	3
3	Módulos de Operação	4
3.1	Inserção de Usuário	4
3.2	Inserção Múltipla de Usuários	6
3.3	Busca de Usuário por ID	8
3.4	Transferência de Saldo entre Contas	10
3.5	Remoção de Usuário e Gerenciamento de Memória	12
4	Finalização e Persistência de Dados	14
4.1	Geração de Relatório e Encerramento	14

1 Introdução

Este documento detalha o desenvolvimento do sistema **Uaibank**, uma aplicação voltada ao gerenciamento de contas de usuários utilizando estruturas de dados dinâmicas (Listas Encadeadas). O objetivo principal é garantir a eficiência nas operações de cadastro, busca, movimentação financeira e persistência de dados em arquivos de texto.

2 Arquitetura Geral do Sistema

A arquitetura do sistema é baseada em um loop principal que gerencia o menu de opções. Cada escolha do usuário aciona um módulo específico, garantindo a modularização do código.

2.1 Pseudocódigo da Função Principal (Main)

INÍCIO

```
// Preparação inicial do sistema
Inicializar banco de dados de usuários vazio
Definir próximo ID disponível como 1
```

REPITA

```
EXIBIR Menu de Opções
LER opcao_escolhida
```

ESCOLHA (opcao_escolhida)

```
CASO 1: Realizar inserção de um novo usuário
CASO 2: Realizar inserção múltipla de usuários
CASO 3: Realizar busca de usuário por ID
CASO 4: Realizar transferência de saldo entre usuários
CASO 5: Realizar remoção de usuário por ID
CASO 0: Gerar_Arquivo_E_Encerrar(lista_de_usuarios)
CASO PADRÃO: EXIBIR "Opção inválida!"
```

FIM_ESCOLHA

```
ATE_QUE (opcao_escolhida == 0)
```

FIM

2.2 Fluxograma da Estrutura Geral

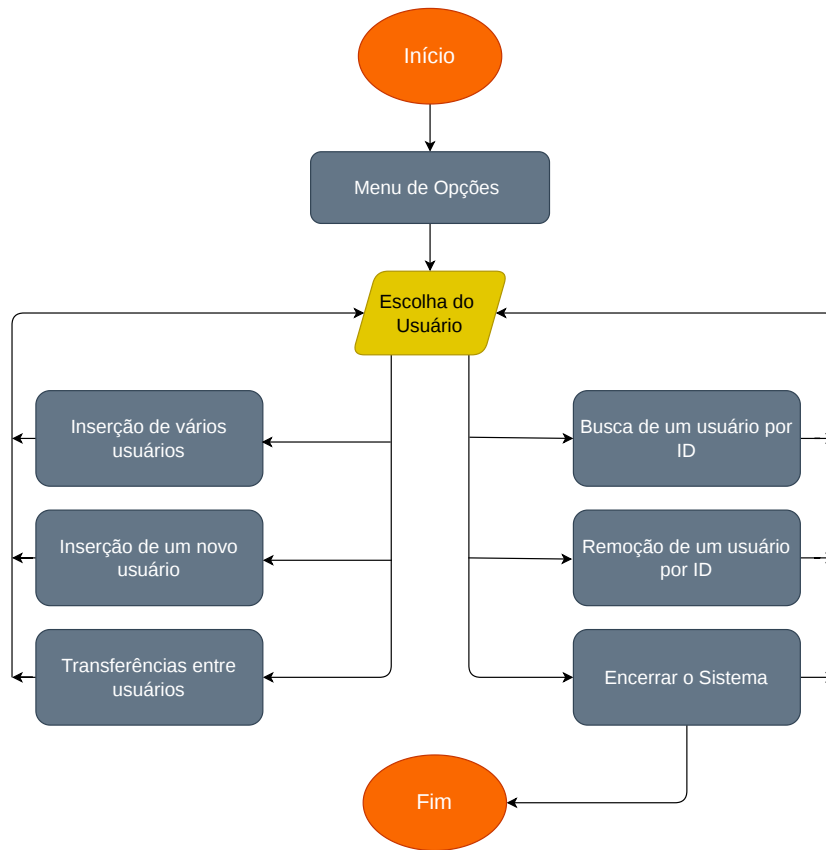


Figura 1: Fluxograma da estrutura principal e menu do sistema.

3 Módulos de Operação

Nesta seção, detalhamos cada uma das funcionalidades oferecidas pelo Uaibank, acompanhadas de sua lógica interna e representação visual.

3.1 Inserção de Usuário

Esta operação é responsável por registrar novos clientes no sistema. Ela realiza o tratamento da string do nome (limitando a 100 caracteres) e valida se o saldo inicial não é negativo antes de alocar o nó na lista encadeada.

Pseudocódigo

```
função Inserir_Novo_Usuario()
INÍCIO
    // Leitura e tratamento do Nome
    LER nome
    SE (quantidade_de_caracteres(nome) > 100) ENTÃO
        nome <- considerar apenas os 100 primeiros caracteres
    FIM_SE

    // Leitura dos demais dados
    LER idade, saldo

    // Validação do saldo
    SE (saldo < 0) ENTÃO
        EXIBIR "Erro: Saldo não pode ser negativo."
    SENÃO
        // Processamento de memória e atribuição
        Alocar espaço de memória para novo_usuario
        novo_usuario.id <- gerador_id
        novo_usuario.nome <- nome
        novo_usuario.idade <- idade
        novo_usuario.saldo <- saldo

        // Atualiza a variável global para o próximo usuário
        gerador_id <- gerador_id + 1

        // Inserção na estrutura de dados
        Adicionar novo_usuario ao final da Lista Encadeada

        // Saída de sucesso
        EXIBIR "Usuário inserido com id ", novo_usuario.id
    FIM_SE
FIM
```

Fluxograma de Inserção

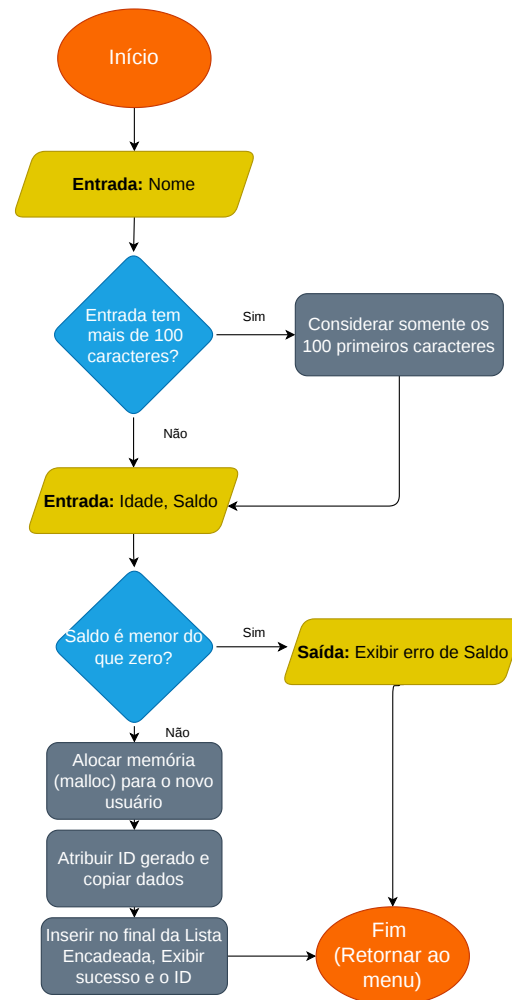


Figura 2: Fluxo lógico da função de inserção de usuários.

3.2 Inserção Múltipla de Usuários

Este módulo permite a automatização do cadastro, solicitando ao usuário uma quantidade específica (N) de registros. O sistema então executa o ciclo de inserção repetidamente, reaproveitando a lógica de validação do módulo de inserção individual.

Pseudocódigo

função Insercao_Multipla()

INÍCIO

 // Leitura da quantidade de repetições

 LER quantidade_N

 contador <- 0

 // Laço de repetição para inserir N usuários válidos

 ENQUANTO (contador < quantidade_N) FAÇA

 // Executa a função e guarda o resultado

 resultado <- EXECUTAR função Inserir_Novo_Usuario()

 // Só incrementa o contador se o cadastro não teve erros

 SE (resultado == SUCESSO) ENTÃO

 Incrementar +1 no contador

 SENÃO

 EXIBIR "Erro no cadastro. Por favor, insira os dados novamente."

 FIM_SE

 FIM_ENQUANTO

FIM

Fluxograma de Inserção Múltipla

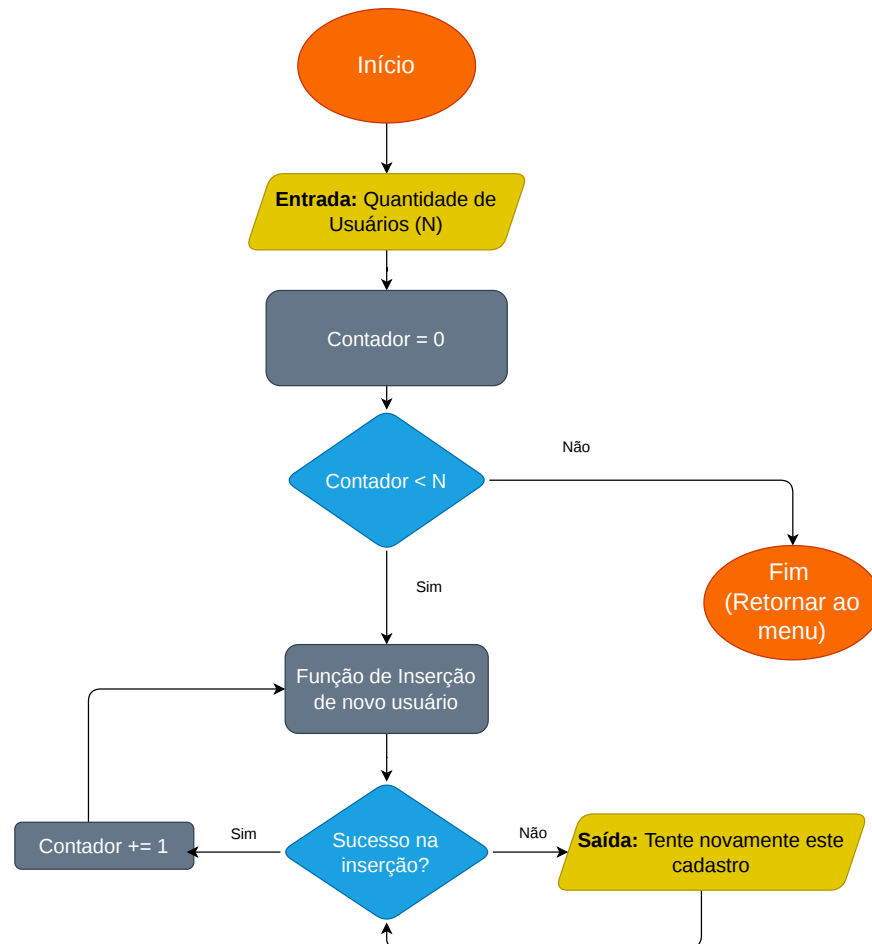


Figura 3: Representação do laço de repetição para inserções em lote.

3.3 Busca de Usuário por ID

Este módulo realiza uma busca linear na lista encadeada. Ele percorre cada nó a partir do início, comparando o identificador fornecido pelo usuário com os IDs armazenados, até encontrar uma correspondência ou atingir o fim da estrutura.

Pseudocódigo

função Buscar_Usuario_Por_ID()

INÍCIO

 LER ID_buscado

 // Define o ponto de partida

 usuario_atual <- primeiro_usuario_da_lista

 ENQUANTO (Verdadeiro) FAÇA

 SE (usuario_atual chegou ao fim) ENTÃO

 EXIBIR "Erro: Usuário não encontrado."

 RETORNAR ao Menu

 FIM_SE

 SE (usuario_atual.id == ID_buscado) ENTÃO

 EXIBIR usuario_atual.nome, usuario_atual.idade, usuario_atual.saldo

 RETORNAR ao Menu

 SENÃO

 usuario_atual <- usuario_atual.proximo

 FIM_SE

 FIM_ENQUANTO

FIM

Fluxograma de Busca

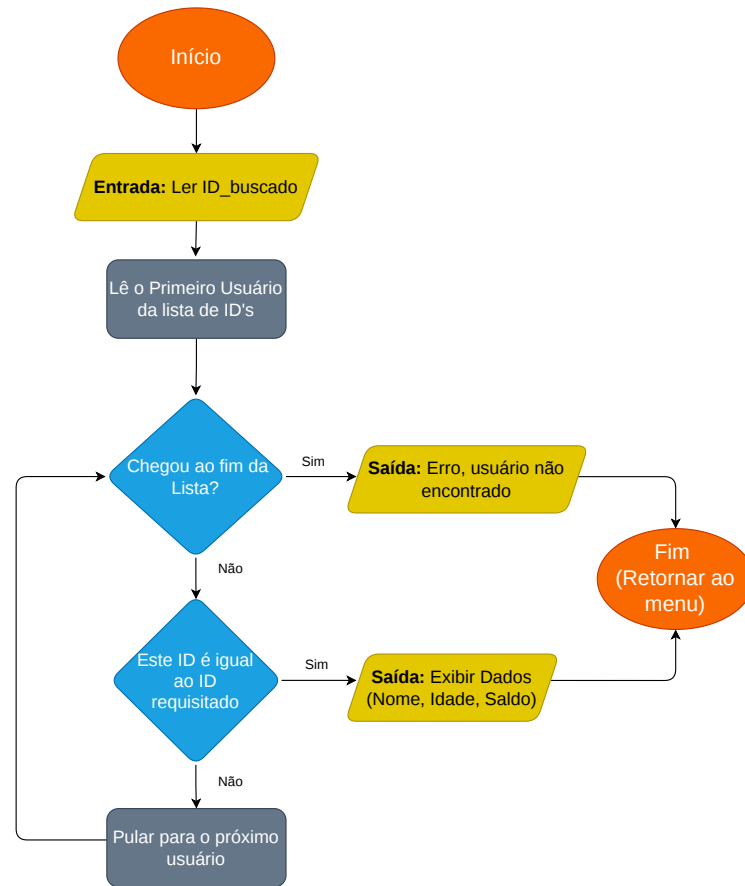


Figura 4: Fluxo de busca sequencial na lista encadeada.

3.4 Transferência de Saldo entre Contas

Este módulo é um dos mais críticos do sistema, pois envolve a manipulação de dois nós distintos da lista encadeada simultaneamente. Ele realiza a busca de ambos os envolvidos (pagador e recebedor) e aplica as regras de negócio: verificação de existência das contas e validação de saldo suficiente para a operação.

Pseudocódigo

```
função Realizar_Transferencia()
INÍCIO
    LER id_pagador, id_recebedor, valor_transferencia

    // 1. Busca e Validação do Pagador
    pagador <- PROCURAR na lista o usuário com ID == id_pagador
    SE (pagador NÃO EXISTE) ENTÃO
        EXIBIR "Erro! Pagador não existe"
        RETORNAR ao Menu
    FIM_SE

    // 2. Busca e Validação do Recebedor
    recebedor <- PROCURAR na lista o usuário com ID == id_recebedor
    SE (recebedor NÃO EXISTE) ENTÃO
        EXIBIR "Erro! Recebedor não existe"
        RETORNAR ao Menu
    FIM_SE

    // 3. Validação de valor positivo
    SE (valor < 0) ENTÃO
        EXIBIR "Erro! Valor Negativo"
        RETORNAR ao Menu

    // 4. Validação de Regra de Negócio (Saldo)
    SE (pagador.saldo < valor_transferencia) ENTÃO
        EXIBIR "Erro! Saldo insuficiente"
        RETORNAR ao Menu
    SENÃO
        // 4. Efetivação da transação
        pagador.saldo <- pagador.saldo - valor_transferencia
        recebedor.saldo <- recebedor.saldo + valor_transferencia

        EXIBIR "Transferência realizada com sucesso"
    FIM_SE
FIM
```

Fluxograma de Transferência

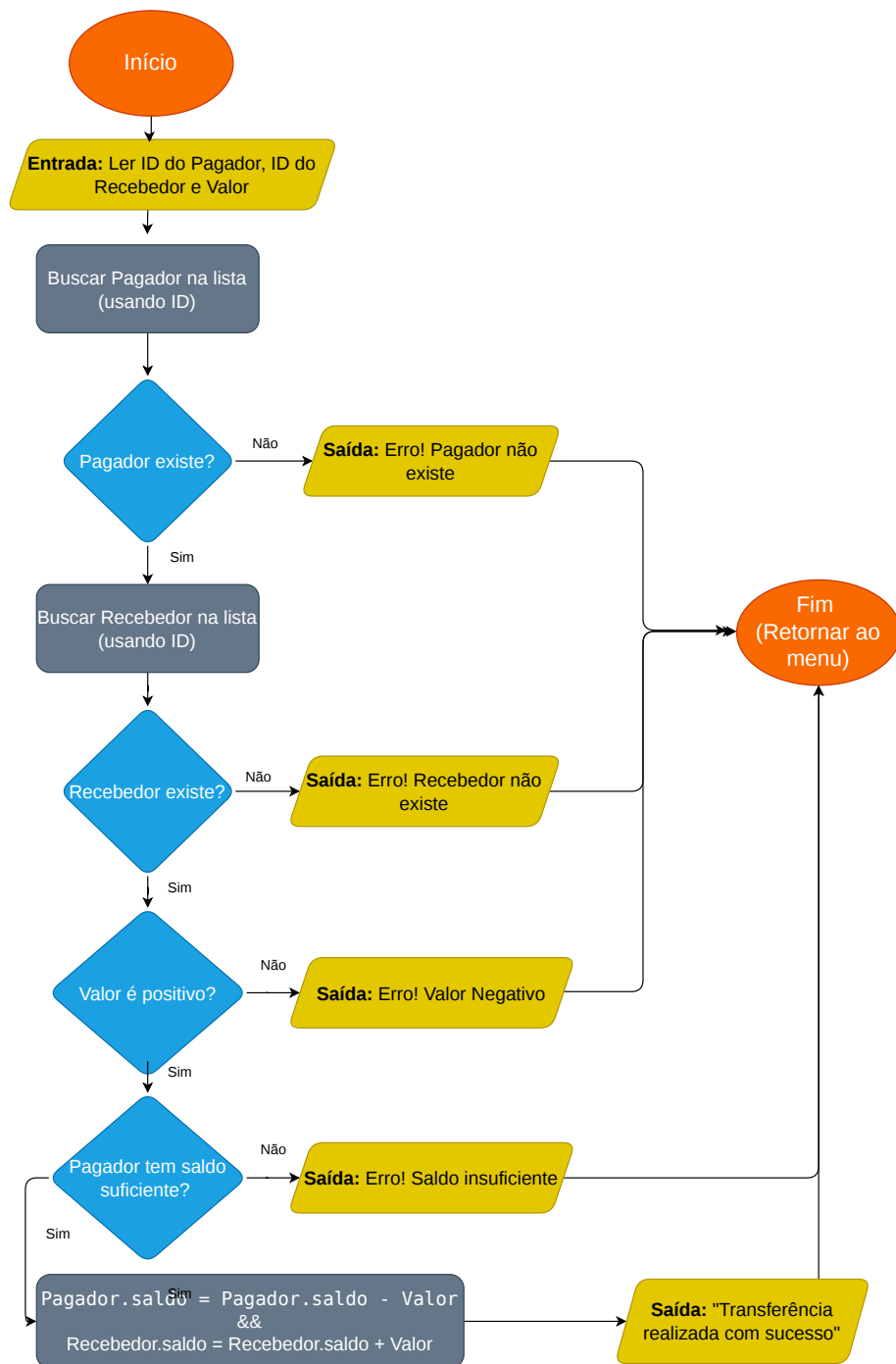


Figura 5: Fluxograma detalhado da lógica de transferência bancária entre usuários.

3.5 Remoção de Usuário e Gerenciamento de Memória

Este módulo trata da exclusão de registros e da manutenção da integridade da lista encadeada. Além de localizar o usuário e ajustar os ponteiros dos nós adjacentes para evitar a quebra da estrutura, o sistema realiza a liberação do espaço em memória RAM, prevenindo vazamentos de memória (*memory leaks*).

Pseudocódigo

```
função Remover_Usuario()
INÍCIO
    LER id_remove

    // Procura o usuário na estrutura
    usuario_alvo <- PROCURAR na lista o usuário com ID == id_remove

    SE (usuario_alvo NÃO EXISTE) ENTÃO
        EXIBIR "Erro! Usuário não encontrado."
        RETORNAR ao Menu
    SENÃO
        // Remove da fila e apaga os dados da memória
        DESCONECTAR usuario_alvo da lista (atualizar vizinhos)
        LIBERAR_MEMORIA (usuario_alvo)

        EXIBIR "Usuário removido com sucesso."
    FIM_SE
FIM
```

Fluxograma de Remoção

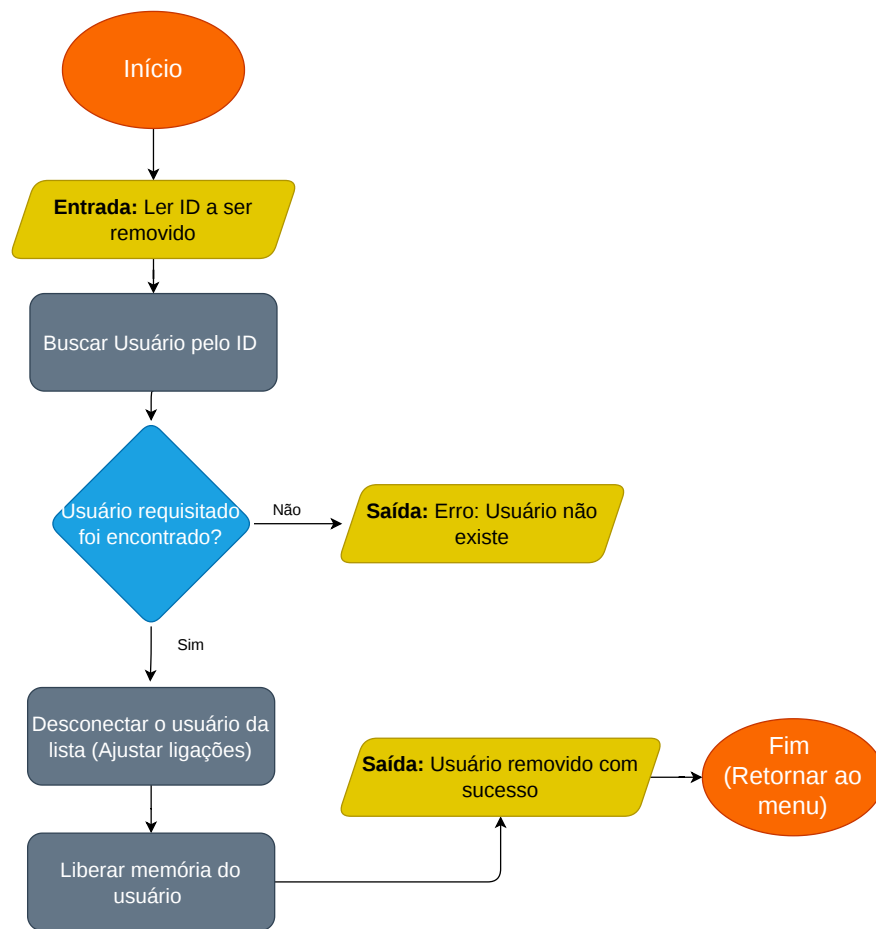


Figura 6: Lógica de desconexão de nós e desalocação de memória.

4 Finalização e Persistência de Dados

A etapa final do sistema garante que todas as alterações feitas na memória RAM sejam persistidas em um arquivo físico. Este processo é vital para que os dados dos usuários não sejam perdidos após o encerramento da execução.

4.1 Geração de Relatório e Encerramento

Este módulo percorre a lista encadeada uma última vez, exportando os dados de cada nó para um arquivo de texto (`relatorio.txt`) e, em seguida, finaliza a execução do software de forma segura.

Pseudocódigo

```
função Gerar_Arquivo_E_Encerrar(lista_de_usuarios)
INÍCIO
    arquivo <- ABRIR arquivo "relatorio.txt" no modo escrita ("w")

    SE (arquivo == ERRO) ENTÃO
        EXIBIR "Erro: Não foi possível criar o arquivo."
        RETORNAR
    FIM_SE

    usuario_atual <- lista_de_usuarios

    ENQUANTO (usuario_atual NÃO É NULO) FAÇA
        ESCREVER_NO_ARQUIVO (arquivo, usuario_atual.id,
                               usuario_atual.nome, usuario_atual.idade,
                               usuario_atual.saldo)
        usuario_atual <- usuario_atual.proximo
    FIM_ENQUANTO

    FECHAR arquivo
    EXIBIR "Relatório gerado com sucesso!"
FIM
```

Fluxograma de Persistência

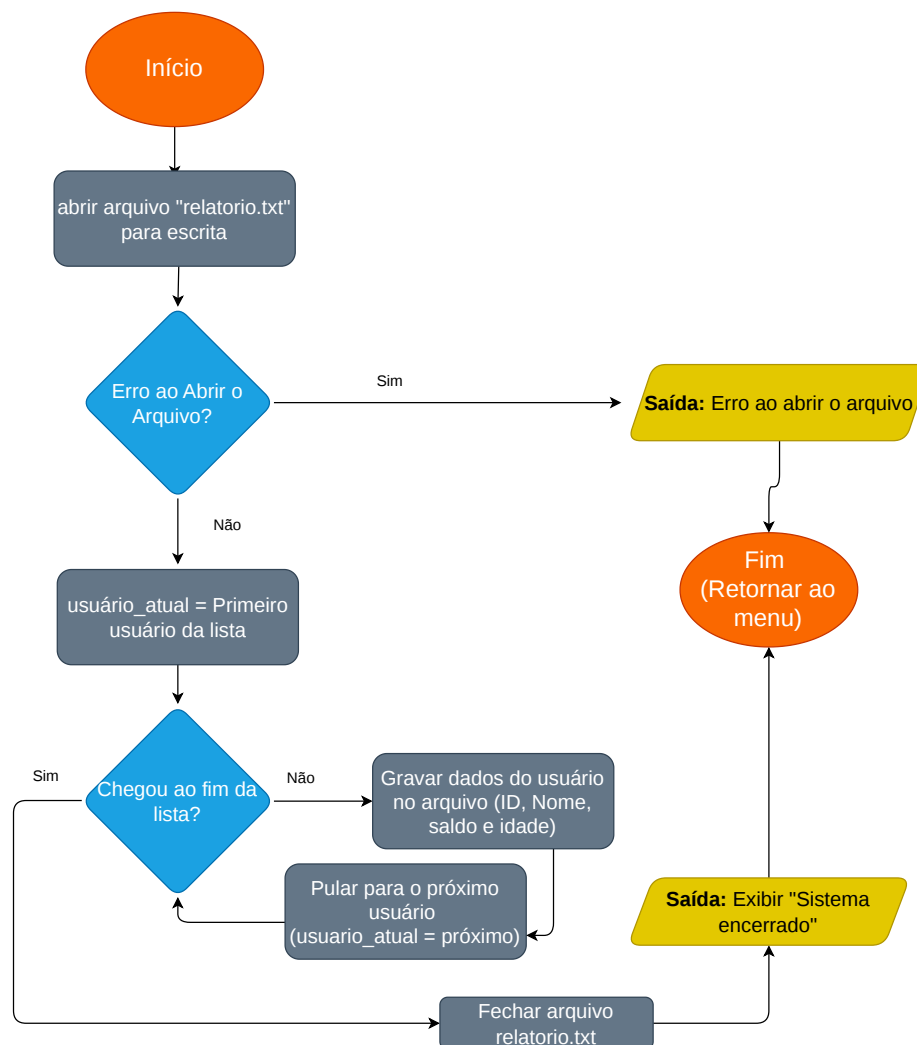


Figura 7: Processo de iteração final para escrita em arquivo e encerramento.